EXPERIMENT - 44

1. Problem Statement

Write a menu-driven C program to perform the following operations using separate functions:

- Change the date format from dd/mm/yyyy to dd-mmm-yy. For example, the input 12/10/2026 gives the output 12-Oct-26.
- Print all sub-strings of a string. For example, the input abc gives the output a, ab, abc, b, bc, c.

2. Algorithm

Main Program

Step 1: START

Step 2: Display the menu and read choice.

Step 3: Using a switch statement on choice:

If choice is 1, read a date, call changeDateFormat() and display the string it fills in.

If choice is 2, read a string and call printSubstrings().

If choice is 3, display "Exiting the program."

Step 4: Repeat from Step 2 while choice is not equal to 3.

Step 5: STOP

Function changeDateFormat()

Step 1: Store the twelve short month names Jan to Dec in the array months.

Step 2: Read dd, mm and yyyy out of the date using sscanf() with the format "%d/%d/%d".

Step 3: Build newDate using sprintf() as dd, then a hyphen, then months[mm − 1], then a hyphen, then yyyy % 100, with the day and the year printed in two digits.

Step 4: Return to the calling function.

Function printSubstrings()

Step 1: Set n = strlen(s) and first = 1.

Step 2: Repeat for i = 0 to n − 1:

Repeat for j = i to n − 1:

If first is 0, display a comma.

Display s[k] for k = i to j, then set first = 0.

Step 3: Move to a new line.

Step 4: Return to the calling function.

3. Test Case Table

#	Operation	Input	Expected Output	Actual Output	Result
1	1. Date format	12/10/2026	12-Oct-26, the example from the question	12-Oct-26	**PASS**
2	1. Date format	05/01/2005	05-Jan-05, with the leading zeros kept	05-Jan-05, both zeros still there	**PASS**
3	1. Date format	31/12/1999	31-Dec-99, so a year before 2000 works too	31-Dec-99	**PASS**
4	1. Date format	09/09/2009	09-Sep-09	09-Sep-09, month 9 gave Sep and not Oct	**PASS**
5	2. Sub-strings	abc	a,ab,abc,b,bc,c, the example from the question	Matches the question exactly	**PASS**
6	2. Sub-strings	code	Ten sub-strings, from c up to code	c,co,cod,code,o,od,ode,d,de,e	**PASS**
7	2. Sub-strings	ab	a,ab,b	Three of them, as expected	**PASS**
8	2. Sub-strings	a	Just a, with no comma anywhere	a, and the line ended cleanly	**PASS**
9	3. Exit	choice = 3	Exiting the program, and the loop ends	Loop ended	**PASS**

4. Source Code

strops44.c

```
/* Experiment 42. One function for each of the three operations.
   Every option reads its own input here, which is different from what
   I have been doing. Option 3 needs two strings and the other two need
   one, so there was no single string I could read at the start that
   would have suited all three.
   The question lists only three operations, but a do while menu has no
   way of stopping, so option 4 is mine. Without it the menu keeps
   coming back and the only way out is to kill the program. */

#include <stdio.h>

void removeVowels(const char s[], char result[]);
char firstRepeatingLowercase(const char s[]);
int areAnagrams(const char s1[], const char s2[]);

int main(void)
{
    char str[100], str2[100], result[100];
    char ch;
    int choice;

    do {
        printf("\nMENU\n");
        printf("1. Remove all the vowels from a string\n");
        printf("2. Find the first repeating lowercase letter\n");
        printf("3. Check whether two strings are anagrams\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter a string: ");
                scanf(" %[^\n]", str);
                removeVowels(str, result);
                printf("String without vowels: %s\n", result);
                break;

            case 2:
                printf("Enter a string: ");
                scanf(" %[^\n]", str);
                ch = firstRepeatingLowercase(str);
                if (ch == '\0')
                    printf("No lowercase letter repeats in this string\n");
                else
                    printf("First repeating lowercase letter: %c\n", ch);
                break;

            case 3:
                printf("Enter the first string: ");
                scanf(" %[^\n]", str);
                printf("Enter the second string: ");
                scanf(" %[^\n]", str2);
                if (areAnagrams(str, str2))
                    printf("The two strings are anagrams\n");
                else
                    printf("The two strings are not anagrams\n");
                break;

            case 4:
                printf("Exiting the program.\n");
                break;
        }
    } while (choice != 4);

    return 0;
}
```

```
/* i walks through the old string and j marks where to write in the new
   one, and they drift apart because j only moves when something is
   actually copied.
   The '\0' on the last line is the one I forgot. Programming in C came
   out right and then had junk stuck on the end of it, because printf
   had no idea where the new string finished.
   Both cases are listed because A and a are different characters, and
   leaving the capitals out would have let the A through in Day. */
void removeVowels(const char s[], char result[])
{
    int i, j = 0;
    char ch;

    for (i = 0; s[i] != '\0'; i++) {
        ch = s[i];

        if (ch != 'a' && ch != 'e' && ch != 'i' && ch != 'o' && ch != 'u' &&
            ch != 'A' && ch != 'E' && ch != 'I' && ch != 'O' && ch != 'U') {
            result[j] = ch;
            j++;
        }
    }

    result[j] = '\0';
}

/* First repeating can be read two ways, and the two give different
   answers. In abcdbca one reading says b, the first letter you meet
   that you have seen before. The other says a, because out of the
   letters that repeat at all, a is the one that showed up earliest.
   I went with the second, which is the usual meaning, and that is why
   the counting happens first and the walk from the left happens after.
   One single pass would have given the other answer.
   count has 26 boxes, one per small letter, and s[i] - 'a' turns a
   letter straight into its box number, since 'a' is 97 and 'b' is 98.
   Capitals are skipped in both loops, so AABBcdc answers c, even though
   A and B repeat more times than c does. */
char firstRepeatingLowercase(const char s[])
{
    int count[26] = {0};
    int i;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z')
            count[s[i] - 'a']++;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z' && count[s[i] - 'a'] > 1)
            return s[i];

    /* '\0' is not a letter, so main can use it to mean nothing was
       found. */
    return '\0';
}

/* Anagrams hold the same characters the same number of times, only in
   a different order. My first idea was to sort both strings and see if
   they came out identical, but counting turned out to be less code, and
   it leaves the two strings alone instead of rearranging them.
   256 boxes covers every character there is, so spaces and digits are
   counted along with the letters, and nothing is quietly ignored.
   The length test is not only a shortcut. The counts could agree on
   every character and the strings still be different lengths if one of
   them held something the other did not. */
int areAnagrams(const char s1[], const char s2[])
{
    int count1[256] = {0};
    int count2[256] = {0};
    int i, len1 = 0, len2 = 0;

    for (i = 0; s1[i] != '\0'; i++) {
        count1[(unsigned char) s1[i]]++;
        len1++;
```

```
    }

    for (i = 0; s2[i] != '\0'; i++) {
        count2[(unsigned char) s2[i]]++;
        len2++;
    }

    if (len1 != len2)
        return 0;

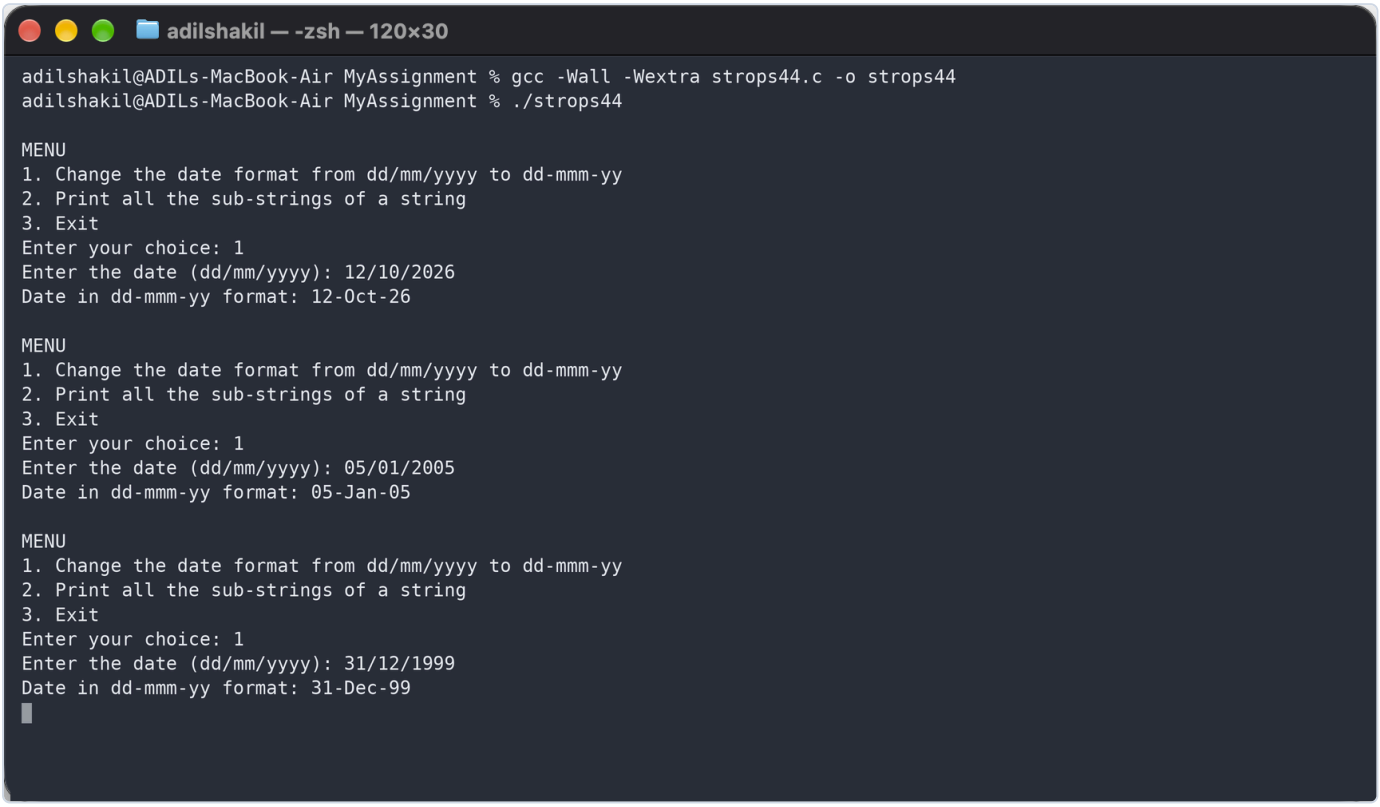
    for (i = 0; i < 256; i++)
        if (count1[i] != count2[i])
            return 0;

    return 1;
}
```

5. Compilation and Output

I compiled it with `gcc -Wall -Wextra strops44.c -o strops44`. Those two flags show up even the small warnings, and it built clean.

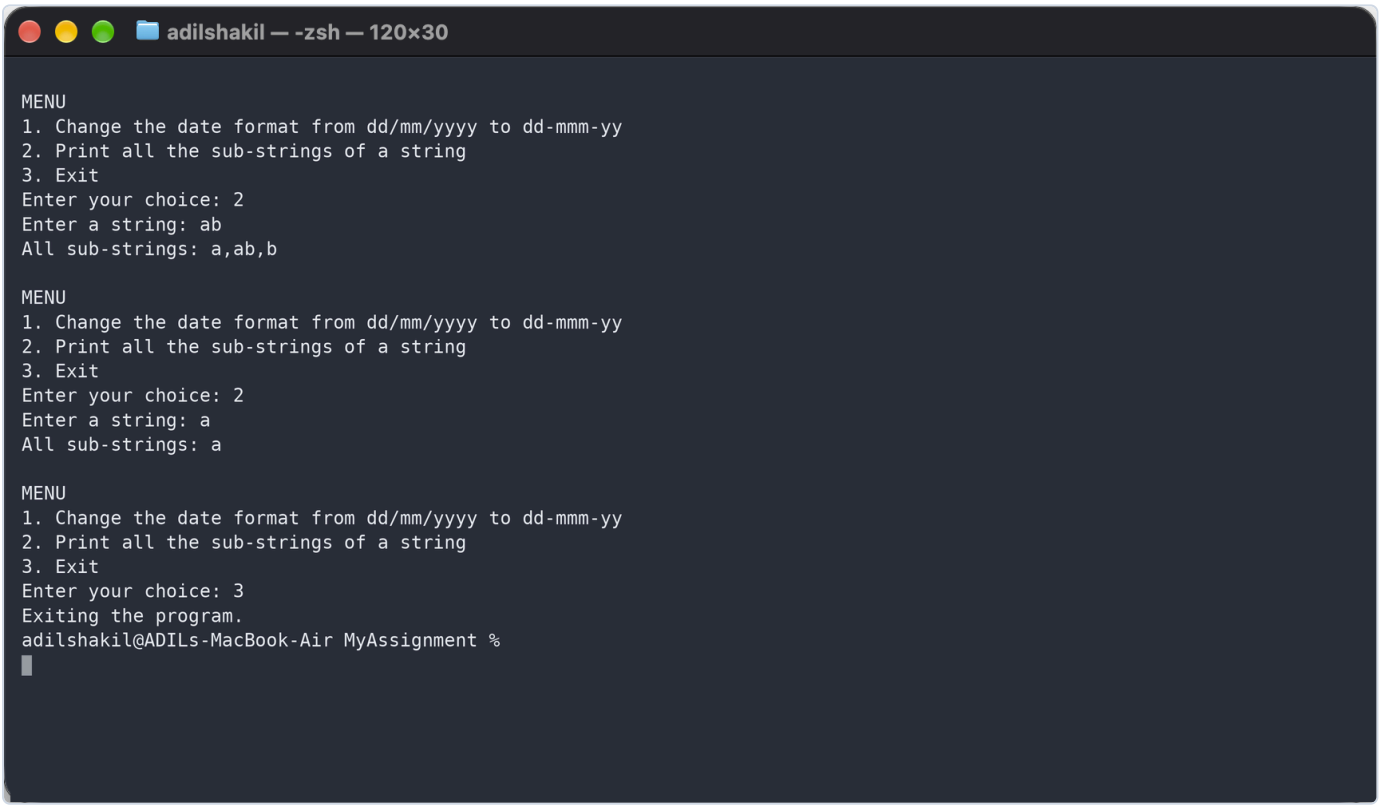
Three Dates in the New Format



One More Date, and the Sub-strings of abc and code



A Two Letter String, a Single Letter, and Exit



All nine cases came out right, and cases 1 and 5 match the two examples printed in the question exactly.

The date comes in as one string, so the first job is getting the three numbers out of it. sscanf does that in a line. It works like scanf except that it reads from a string instead of from the keyboard, and because the slashes are written into the format, it skips over them and drops the numbers straight into dd, mm and yyyy.

The mistake I made there was with the months array. A list in C starts counting at 0, so Jan is at position 0 and October is at position 9, not 10. I wrote `months[mm]` and 12/10/2026 came out as 12-Nov-26, which looked close enough to be confusing. It has to be `months[mm - 1]`.

The rest of the line is sprintf, which builds a string the same way printf builds a line of output, only it writes into newDate instead of on the screen. `%02d` is what keeps the zero on a day like 05, and `yyyy % 100` drops the century, which is how 2026 becomes 26 and 1999 becomes 99.

For option 2 the thing to see is that a sub-string is just a run of characters sitting together, so one is completely decided by where it starts and where it ends. That is where the two loops come from, i for the start and j for the end, and j starting at i rather than at i + 1 is what lets the single letters in. The third loop only does the printing.

The commas gave me more trouble than the loops did. Printing one after every sub-string left a stray comma on the end of the line, so the flag is there to make the comma come before each one except the very first. Case 8 is the test for that: a string of one letter prints just a, with nothing hanging off it.

Last thing. The question lists two operations and no exit, but a menu built on a do while loop has nothing to stop it, so option 3 is mine.